



Sistema de Gestão de Eventos e Finanças Pessoais

Disciplina

Desenvolvimento de Sistemas para Web

Professor

Wanderson Muniz de Santana

Integrantes

Caio Drumond de Abreu Rebouças

Matrícula: 202416622

Cecília Magalhães Santos

Matrícula: 202414237

João Victor Sanchez Gonçalves Furtado

Matrícula: 202414198

<https://timecash.onrender.com>

github.com/caiakos/timecash

1. Descrição Geral do Produto

O TimeCash é uma aplicação web voltada para o controle de eventos e finanças pessoais. A ideia surgiu da necessidade de reunir, em um único lugar, o planejamento de compromissos e o acompanhamento de entradas e saídas financeiras — algo que geralmente fica espalhado entre agendas, planilhas e aplicativos separados.

Na prática, o usuário consegue criar uma conta, registrar eventos com data, horário, categoria e local, lançar transações financeiras (entradas e saídas) e visualizar tudo em um calendário mensal interativo. As finanças podem ser vinculadas a eventos específicos, facilitando o controle de gastos por ocasião.

Funcionalidades principais:

Autenticação: Cadastro e login com validação de e-mail, senha criptografada e proteção de rotas.

Calendário: Visualização mensal com navegação entre meses, exibindo eventos e resumo financeiro do período.

Gestão de Eventos: Criação, edição e exclusão de eventos com título, descrição, data, horário, categoria e local.

Gestão de Finanças: Registro de entradas e saídas com valor, data, descrição e vínculo opcional a eventos.

Resumo Financeiro: Exibição de total de entradas, saídas, saldo do mês e gastos futuros em tempo real.

Multi-usuário: Cada usuário vê e gerencia apenas seus próprios dados, com isolamento total entre contas.

2. Diagrama do Banco de Dados

O banco de dados possui três tabelas principais com os seguintes relacionamentos: um usuário pode ter vários eventos e várias finanças; uma finança pode estar vinculada a um evento (opcional). A exclusão de um usuário remove em cascata todos os seus dados.

Tabela: users

Coluna	Tipo	Restrições
id	Integer	PK, auto-increment
name	String(100)	NOT NULL
email	String(150)	NOT NULL, UNIQUE
password_hash	String(256)	NOT NULL
created_at	DateTime	default: now()

Tabela: events

Coluna	Tipo	Restrições
id	Integer	PK, auto-increment
title	String(150)	NOT NULL
description	Text	nullable
date	Date	NOT NULL
time	Time	nullable
category	String(50)	NOT NULL, default: Geral
location	String(200)	nullable
created_at	DateTime	default: now()
user_id	Integer	FK -> users.id, NOT NULL

Tabela: finances

Coluna	Tipo	Restrições
id	Integer	PK, auto-increment
description	String(200)	NOT NULL
amount	Float	NOT NULL
type	String(10)	NOT NULL ('entrada' ou 'saida')
date	Date	NOT NULL
created_at	DateTime	default: now()
user_id	Integer	FK -> users.id, NOT NULL
event_id	Integer	FK -> events.id, nullable

Relacionamentos:

- users (1) — (N) events | cascade: all, delete-orphan
- users (1) — (N) finances | cascade: all, delete-orphan
- events (1) — (N) finances | event_id é opcional (nullable)

3. Detalhamento Técnico do Código

3.1 Application Factory — app/__init__.py

A aplicação usa o padrão Application Factory, onde toda a inicialização acontece dentro da função `create_app()`. Isso permite criar instâncias isoladas do app para testes e facilita a configuração por ambiente (desenvolvimento, produção).

```
def create_app(): app = Flask(__name__) database_url = os.getenv('DATABASE_URL',
'sqlite:///timecash.db') if database_url.startswith('postgres://'): database_url =
database_url.replace('postgres://', 'postgresql://', 1)
app.config['SQLALCHEMY_DATABASE_URI'] = database_url db.init_app(app)
login_manager.init_app(app) # Registro dos blueprints
app.register_blueprint(auth_bp, url_prefix='/auth')
app.register_blueprint(events_bp, url_prefix='/eventos')
app.register_blueprint(finances_bp, url_prefix='/financas')
app.register_blueprint(calendar_bp, url_prefix='/calendario') with
app.app_context(): db.create_all() return app
```

O trecho de correção da URL (`postgres://` → `postgresql://`) foi necessário porque o Render.com fornece a string de conexão no formato antigo, incompatível com versões recentes do SQLAlchemy.

3.2 Modelos — app/models.py

Os três modelos herdam de `db.Model` e definem as tabelas via ORM. O modelo `User` usa `UserMixin` do Flask-Login e armazena senhas com hash via Werkzeug. Os relacionamentos usam `cascade` para garantir integridade referencial automática.

```
class Event(db.Model): __tablename__ = 'events' id = db.Column(db.Integer,
primary_key=True) title = db.Column(db.String(150), nullable=False) date =
db.Column(db.Date, nullable=False) category = db.Column(db.String(50),
nullable=False, default='Geral') user_id = db.Column(db.Integer,
db.ForeignKey('users.id'), nullable=False) finances = db.relationship('Finance',
backref='event', cascade='all, delete-orphan') @property def total_gastos(self):
return sum(f.amount for f in self.finances if f.type == 'saida')
```

3.3 Formulários e Validação — auth/forms.py, events/forms.py

A validação é feita no servidor com Flask-WTF e WTForms. Cada formulário declara seus campos e validators. O formulário de registro inclui validação customizada para impedir cadastro com e-mail já existente no banco.

```
class RegisterForm(FlaskForm): name = StringField('Nome',
validators=[DataRequired(), Length(min=3)]) email = StringField('Email',
validators=[DataRequired(), Email()]) password = PasswordField('Senha',
validators=[DataRequired(), Length(min=6)]) confirm = PasswordField('Confirmar',
validators=[EqualTo('password')]) def validate_email(self, field): if
User.query.filter_by(email=field.data).first(): raise ValidationError('Este email
já está cadastrado.')
```

3.4 Calendário — calendar/routes.py

A rota do calendário recebe mês e ano via query string, corrige navegação nos limites (ex: mês 0 vira dezembro do ano anterior), busca eventos e finanças do período e agrupa os eventos

por dia. Uma proteção foi adicionada para lidar com datas retornadas como string pelo SQLite.

```
events_by_day = {} for e in events: event_date = e.date if isinstance(event_date,
str): event_date = datetime.strptime(event_date, '%Y-%m-%d').date() day =
event_date.day if day not in events_by_day: events_by_day[day] = []
events_by_day[day].append(e)
```

4. Arquitetura Adotada

O projeto segue o padrão MVC (Model-View-Controller) adaptado para Flask, com separação clara entre as camadas de dados, lógica e apresentação. A organização em Blueprints permite que cada módulo funcione de forma independente.

Camada	Onde fica	Responsabilidade
Model	app/models.py	Definição das entidades e acesso ao banco via ORM
View	app/templates/	Templates Jinja2 com herança de base.html
Controller	app/*/routes.py	Lógica das rotas, validação e redirecionamentos
Forms	app/*/forms.py	Declaração e validação dos formulários WTForms
Config	app/__init__.py	Application Factory, extensões e blueprints

Estrutura de pastas:

```
timecash/ ■■■ app/ ■ ■■■ __init__.py # Application Factory ■ ■■■ models.py #
User, Event, Finance ■ ■■■ auth/ # Blueprint de autenticação ■ ■ ■■■ routes.py
■ ■ ■■■ forms.py ■ ■■■ events/ # Blueprint de eventos ■ ■■■ finances/ #
Blueprint de finanças ■ ■■■ calendar/ # Blueprint do calendário ■ ■■■ templates/
# HTML (Jinja2) ■ ■■■ static/ # CSS e JS ■■■ tests/ # Testes automatizados ■■■
run.py # Ponto de entrada ■■■ requirements.txt ■■■ .env
```

5. Tecnologias Utilizadas

Tecnologia	Versão	Uso no projeto
Flask	3.0.3	Framework web principal
Flask-SQLAlchemy	3.1.1	ORM para acesso ao banco de dados
Flask-Login	0.6.3	Gerenciamento de sessão e autenticação
Flask-WTF / WTForms	1.2.1 / 3.1.2	Formulários com validação no servidor
Werkzeug	3.0.3	Hash de senhas e utilitários HTTP
Bootstrap	5.3.3	Framework CSS para responsividade e UI
PostgreSQL	-	Banco de dados em produção (Render.com)
SQLite	-	Banco de dados local para desenvolvimento e testes
Gunicorn	21.2.0	Servidor WSGI para produção
pytest / pytest-flask	8.2.2	Testes automatizados
python-dotenv	1.0.1	Gerenciamento de variáveis de ambiente
Render.com	-	Hospedagem da aplicação em produção

Flask foi escolhido por ser o framework trabalhado em aula, pela sua leveza e pela flexibilidade de organização em Blueprints. O PostgreSQL foi adotado em produção por ser o banco relacional gratuito disponível na plataforma Render, substituindo o SQLite que é usado apenas localmente. O Bootstrap foi incluído para garantir responsividade e atender ao critério de uso de framework CSS.

6. Como Rodar o Projeto

Pré-requisitos:

- Python 3.10 ou superior instalado
- Git instalado (opcional)

Passo a passo:

1. Clonar ou baixar o projeto

```
git clone https://github.com/caiakos/timecash.git cd timecash
```

2. Criar e ativar o ambiente virtual

```
python -m venv venv # Windows: venv\Scripts\activate # Linux/Mac: source  
venv/bin/activate
```

3. Instalar as dependências

```
pip install -r requirements.txt
```

4. Configurar as variáveis de ambiente

```
# Crie um arquivo .env na raiz com: SECRET_KEY=sua-chave-aqui  
DATABASE_URL=sqlite:///timecash.db FLASK_ENV=development
```

5. Rodar a aplicação

```
python run.py
```

6. Acessar no navegador

```
http://localhost:5000
```

7. Rodar os testes (opcional)

```
pytest
```

A aplicação também está disponível online em <https://timecash.onrender.com> — basta criar uma conta para começar a usar.

7. Principais Desafios e Como Foram Resolvidos

7.1 Eventos não apareciam no calendário

Problema: Ao rodar o projeto no Google Colab com SQLite, as datas eram retornadas como strings em vez de objetos date do Python. A comparação `e.date.day` falhava silenciosamente, resultando em `events_by_day` sempre vazio.

Solução: Adicionamos uma verificação de tipo antes de acessar o atributo `.day`: se a data for string, ela é convertida com `datetime.strptime()` antes do agrupamento.

7.2 Navegação entre meses quebrava a aplicação

Problema: Quando o usuário navegava para mês 0 (janeiro - 1), a rota tentava criar `date(ano, 0, 1)`, o que levanta um `ValueError` no Python antes de qualquer correção ser aplicada.

Solução: Movemos a lógica de correção (`mes < 1 → mes = 12, ano -= 1`) para antes de qualquer uso do valor de mês nas queries do banco.

7.3 Deploy falhou com psycopg2 no Python 3.14

Problema: O Render alocou Python 3.14 para o projeto e a versão `psycopg2-binary==2.9.9` não era compatível, gerando `ImportError` com símbolo indefinido na biblioteca C.

Solução: Atualizamos para `psycopg2-binary==2.9.10`, que já tem suporte compilado para versões mais recentes do Python.

7.4 SQLAlchemy rejeitava a URL do PostgreSQL do Render

Problema: O Render fornece a `DATABASE_URL` no formato `postgres://`, mas versões recentes do SQLAlchemy exigem o prefixo `postgresql://`.

Solução: Incluímos um tratamento no `create_app()` que detecta e substitui o prefixo automaticamente antes de passar a URL para a configuração do SQLAlchemy.

7.5 Desenvolvimento sem acesso ao terminal local

Problema: Por limitações do ambiente de desenvolvimento, o projeto foi desenvolvido inteiramente pelo Google Colab, sem acesso ao VS Code ou CMD.

Solução: Utilizamos o Colab para escrever e testar o código, o ngrok para expor o servidor localmente durante o desenvolvimento, e o GitHub como repositório central para manter o código sincronizado e fazer deploys no Render.